

Desarrollo de Sistemas Móviles

Sesión 10: Testing, Debugging y Monitoreo de Rendimiento

JUnit 5 · MockK · Turbine · Compose Testing · Profiler · Crashlytics

Al finalizar la clase, serás capaz de:

Pirámide de testing: distribuir correctamente unit, integration y UI tests.

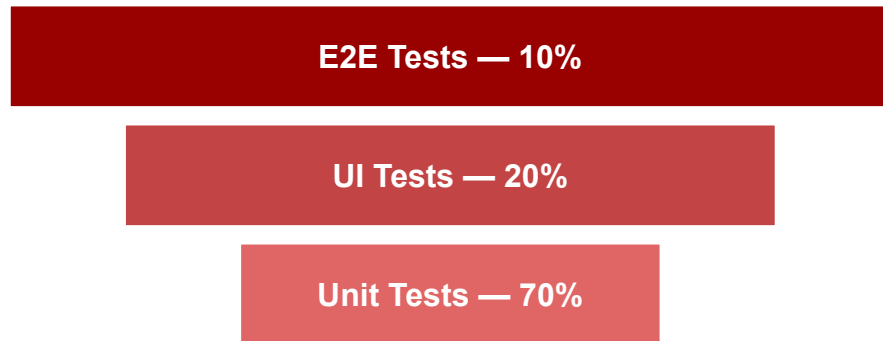
JUnit 5 + MockK: escribir unit tests para ViewModels con coroutines y StateFlow.

Turbine: testear Kotlin Flows y StateFlows de forma declarativa.

Compose Testing API: verificar que los composables renderizan y reaccionan correctamente.

Android Studio Profiler: detectar fugas de memoria y frames lentos (jank).

Firebase Crashlytics: monitorear crashes en producción con contexto de debugging.



E2E:

Espresso / Compose end-to-end

UI:

Compose Testing API

Unit:

JUnit 5, MockK, Turbine

Nivel	Qué testear	Velocidad	Costo
Unit (70%)	ViewModel, UseCase, Repository, mappers	Muy rápido (~ms)	Bajo
Integration (20%)	Room in-memory, Retrofit + MockWebServer	Medio (~s)	Medio
UI / E2E (10%)	Flujos críticos completos	Lento (~min)	Alto

Los unit tests son los más valiosos: rápidos, estables y detectan bugs antes de correr la app.

```
// app/build.gradle.kts
dependencies {
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.0")
    testImplementation("org.junit.jupiter:junit-jupiter-params:5.11.0")

    testImplementation("io.mockk:mockk:1.13.12")
    testImplementation("app.cash.turbine:turbine:1.1.0")
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")
    testImplementation("com.google.truth:truth:1.4.4")
}

tasks.withType<Test> { useJUnitPlatform() } // necesario para JUnit 5
```

JUnit 5 (Jupiter): @Test, @ParameterizedTest, @BeforeEach, @AfterEach.

MockK: mocking nativo de Kotlin — soporta suspend fun, object, companion object.

Turbine: flow.test { awaitItem() } para testear StateFlow y cualquier Flow.

Truth: assertThat(value).isEqualTo(expected) — errores más legibles que JUnit assert.

```
class ProductosViewModelTest {
    private val repository = mockk<ProductosRepository>()
    private val testDispatcher = StandardTestDispatcher()

    @BeforeEach
    fun setup() { Dispatchers.setMain(testDispatcher) }

    @Test
    fun `cargar productos exitosamente actualiza estado a Success`() = runTest {
        // GIVEN - definir comportamiento del mock
        val esperados = listOf(Producto(1, "Laptop", 5200.0, "Tech"))
        coEvery { repository.getProductos() } returns esperados
        val viewModel = ProductosViewModel(GetProductosUseCase(repository))
        // WHEN + THEN - Turbine verifica los estados emitidos
        viewModel.uiState.test {
            assertThat(awaitItem().isLoading).isTrue()
            testDispatcher.scheduler.advanceUntilIdle()
            val success = awaitItem()
            assertThat(success.productos).isEqualTo(esperados)
        }
    }
}
```

```
@Test
fun `error de red actualiza estado con mensaje de error`() = runTest {
    coEvery { repository.getProductos() } throws IOException("Sin conexión")
    val viewModel = ProductosViewModel(GetProductosUseCase(repository))

    viewModel.uiState.test {
        skipItems(1) // ignorar estado loading inicial
        testDispatcher.scheduler.advanceUntilIdle()
        val errorState = awaitItem()
        assertThat(errorState.error).isNotNull()
    }
}
```

```
// Test parametrizado – misma lógica con múltiples inputs
@ParameterizedTest
@ValueSource(strings = ["Laptop", "LAPTOP", "laptop", "lapt"])
fun `busqueda es case-insensitive`(query: String) = runTest { /* ... */ }
```

```
// Crear mocks
val repo = mockk<ProductosRepository>()
val lista = mockk<List<Producto>>(relaxed = true) // retorna defaults sin configurar

// Configurar comportamiento
coEvery { repo.getProductos() } returns listOf(producto1) // suspend fun
every { repo.getSync() } returns listOf(producto1) // fun normal
coEvery { repo.guardar(any()) } just Runs // suspend fun void

// Verificar llamadas
coVerify(exactly = 1) { repo.getProductos() }

// Capturar argumentos para inspeccionarlos
val slot = slot<Producto>()
coEvery { repo.guardar(capture(slot)) } just Runs
assertThat(slot.captured.nombre).isEqualTo("Laptop")
```

mockk<T>() vs **mockk<T>(relaxed = true)**: sin relaxed, todo método no configurado lanza excepción.

Usar any() para no importar el valor exacto del argumento.

✗ Sin Turbine — difícil y frágil

```
val results = mutableListOf<UiState>()
val job = launch {
    viewModel.uiState.collect {
        results.add(it)
    }
}
delay(100) // espera arbitraria
// puede fallar en CI
job.cancel()
```

✓ Con Turbine — declarativo y determinístico

```
viewModel.uiState.test {
    val loading = awaitItem()
    assertThat(loading.isLoading).isTrue()

    testDispatcher.scheduler
        .advanceUntilIdle()

    val success = awaitItem()
    assertThat(success.productos)
        .hasSize(2)

    cancelAndIgnoreRemainingEvents()
}
```

awaitItem() — espera el siguiente elemento. **skipItems(n)** — ignora los primeros n. **awaitComplete()** / **awaitError()** — verifica el final del Flow.

```
androidTestImplementation("androidx.compose.ui:ui-test-junit4")
debugImplementation("androidx.compose.ui:ui-test-manifest")

@RunWith(AndroidJUnit4::class)
class ProductosScreenTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun `lista muestra los productos correctamente`() {
        composeTestRule.setContent {
            MiAppTheme { ProductosList(listOf(Producto(1, "Laptop", 5200.0, "Tech")), {}) }
        }
        composeTestRule.onNodeWithText("Laptop").assertIsDisplayed()
    }

    @Test
    fun `click en producto dispara el callback`() {
        var clickeado: Int? = null
        composeTestRule.onNodeWithText("Laptop").performClick()
        assertThat(clickeado).isEqualTo(1)
    }
}
```

Para elementos sin texto visible (iconos, botones de imagen), usar testTag:

```
// En el composable de producción
@Composable
fun BotonEliminar(onClick: () -> Unit) {
    IconButton(
        onClick = onClick,
        modifier = Modifier.semantics { testTag = "btn_eliminar" }
    ) {
        Icon(Icons.Default.Delete, contentDescription = "Eliminar")
    }
}

// En el test
composeTestRule.onNodeWithTag("btn_eliminar").performClick()

// Loading indicator — usar contentDescription
composeTestRule.onNodeWithContentDescription("Cargando").assertIsDisplayed()
```

Agrega testTag y contentDescription desde el principio — es difícil hacerlo retroactivamente.

En lugar de pensar en casos individuales, Kotest genera miles de inputs aleatorios y verifica propiedades.

```
testImplementation("io.kotest:kotest-runner-junit5:5.9.1")
testImplementation("io.kotest:kotest-property:5.9.1")

class PrecioTest : FunSpec({
    test("precio con descuento siempre es positivo") {
        forAll(
            Arb.double(0.01, 10000.0), // precio aleatorio
            Arb.double(0.0, 0.5)      // descuento 0%-50% aleatorio
        ) { precio, descuento ->
            aplicarDescuento(precio, descuento) > 0 // siempre positivo
        }
    }

    test("serializar y deserializar produce el mismo objeto") {
        forAll(Arb.string(1..100), Arb.double(0.01, 10000.0)) { nombre, precio -> /*...*/ }
    }
})
```

Kotest ejecuta 1000 combinaciones aleatorias por defecto — encuentra casos límite que no imaginarías.

Síntoma: el gráfico de memoria sube gradualmente y nunca baja al volver de una pantalla. (View → Tool Windows → Profiler)

```
// Causa común: binding no liberado
class MiFragment : Fragment() {
    private var _binding: FragmentMiBinding? = null
    private val binding get() = _binding!!

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null // ✅ evita leak
    }
}
```

Cómo diagnosticar:

1. Profiler → Memory tab.
2. Navegar a la pantalla sospechosa y volver — varias veces.
3. Forzar GC (botón de basura).
4. Si la memoria sigue subiendo → tomar heap dump.
5. Buscar instancias activas de la pantalla — debería haber 0.

Un frame tarda más de 16ms (60fps) u 8ms (120fps) → el usuario ve “tartamudeo” visual. **Regla de oro:** todo trabajo pesado fuera del Main Thread.

```
// ❌ Nunca en el Main Thread
viewModel.viewModelScope.launch {
    val data = heavyJson.parse()           // parsing lento en Main Thread = jank
    _state.value = UiState(data)
}

// ✅ Dispatchers.IO para trabajo pesado
viewModel.viewModelScope.launch {
    val data = withContext(Dispatchers.IO) { heavyJson.parse() }
    _state.value = UiState(data)           // actualizar UI en Main Thread: OK
}
```

Cómo diagnosticar: 1. CPU Profiler → System Trace. 2. Interactúa con la app mientras graba. 3. Busca frames en rojo (> 16ms). 4. Identifica qué función corre en el Main Thread.

```
implementation(platform("com.google.firebase:firebase-bom:33.0.0"))
implementation("com.google.firebase:firebase-crashlytics")

// Reportar excepción no-fatal (app sigue corriendo)
try {
    operacionRiesgosa()
} catch (e: Exception) {
    Firebase.crashlytics.recordException(e)
}

// Agregar contexto para debugging en el dashboard
Firebase.crashlytics.setCustomKey("screen", "ProductosList")
Firebase.crashlytics.log("Usuario intentó eliminar producto $productoId")

// Solo en producción — respetar privacidad del usuario
Firebase.crashlytics.setCrashlyticsCollectionEnabled(!BuildConfig.DEBUG)
```

Fatal crashes se reportan automáticamente — no requieren código extra. **Non-fatal** (recordException) para errores esperados que quieres monitorear. **El dashboard** agrupa crashes por tipo, versión, dispositivo y OS.

Concepto	Clave a recordar
Pirámide de testing	70% unit · 20% integration · 10% UI
JUnit 5	@Test, @ParameterizedTest, @BeforeEach. Requiere useJUnitPlatform() en Gradle.
MockK	coEvery { } returns para suspend funs. coVerify para confirmar llamadas.
Turbine	flow.test { awaitItem() } — testear StateFlow sin delays ni races.
Compose Testing	onNodeWithText().assertIsDisplayed(). Usar testTag para elementos sin texto.
Property-Based / Kotest	forAll(Arb.*) — 1000 inputs aleatorios, encuentra casos límite.
Profiler — Memory	Heap dump si la RAM sube sin GC. Liberar _binding = null en onDestroyView.
Profiler — CPU	Frames rojos > 16ms. Todo I/O en Dispatchers.IO, nunca en Main Thread.
Crashlytics	Fatal: automático. Non-fatal: recordException(). Contexto: setCustomKey().



Lab

Laboratorio, Semana 11 (15)

12:15 pm

Gracias